

PRUEBA PRÁCTICA

CryptoWatch: Plataforma de Monitoreo de Criptomonedas y Alertas de Precio

Práctica de preparación técnica

Propósito: evaluar análisis, diseño, desarrollo y despliegue de una solución basada en microservicios, integración con APIs externas, actualización en tiempo real, uso de Memory Cache nativo de .NET, Unit of Work, SQL Server, pruebas unitarias y control de concurrencia.

Documento Confidencial. Prohibida su reproducción total o parcial sin autorización.

1. Enunciado a resolver

Una empresa requiere implementar una plataforma denominada CryptoWatch para monitorear información pública de mercado de criptomonedas y gestionar alertas de precio configuradas por usuarios internos. La solución no realizará compra, venta, custodia de activos, recomendaciones de inversión ni operaciones financieras. Su finalidad es exclusivamente técnica: consulta, visualización, procesamiento, cacheo y trazabilidad de datos de mercado provenientes de APIs externas.

La plataforma debe integrarse con una API externa de información de criptomonedas para consultar precios, variación de mercado y datos relevantes en tiempo cercano al real. La solución debe permitir crear alertas de precio, evaluar su cumplimiento y garantizar que una alerta no sea procesada más de una vez, incluso si varios procesos intentan evaluarla simultáneamente.

Cada criptomoneda monitoreada cuenta con:

- Identificador externo de la criptomoneda, por ejemplo bitcoin, ethereum o solana.
- Símbolo, por ejemplo BTC, ETH o SOL.
- Nombre.
- Precio actual en USD.
- Variación porcentual en 24 horas.
- Volumen de operación en 24 horas.
- Capitalización de mercado.
- Fecha y hora de última actualización.

Cada alerta cuenta con:

- Criptomoneda asociada.
- Precio objetivo.
- Condición: mayor o igual que / menor o igual que.
- Moneda de referencia: USD.
- Estado: Activa / Cumplida / Anulada.
- Fecha de creación.
- Fecha de cumplimiento, cuando aplique.
- Observación.

Se requiere una solución tecnológica que permita consultar datos públicos de mercado, cachear información de alta lectura, administrar alertas y asegurar consistencia cuando las alertas sean evaluadas de forma concurrente.

2. Objetivo de la Prueba

Diseñar y desarrollar una solución que permita:

- Consultar un listado de criptomonedas de interés usando una API externa.
- Consultar el detalle de una criptomoneda, incluyendo precio actual, variación 24h, volumen y capitalización.
- Registrar, editar y anular alertas de precio.
- Evaluar alertas activas contra el precio actual obtenido desde el servicio de mercado.
- Actualizar un dashboard en tiempo real o tiempo cercano al real, sin necesidad de refrescar manualmente la página.
- Usar Memory Cache nativo de .NET para optimizar consultas frecuentes a datos de mercado.
- Aplicar el patrón Unit of Work en las operaciones transaccionales del microservicio de alertas.
- Garantizar que una alerta cumplida se procese una sola vez, incluso ante evaluaciones concurrentes.
- Exponer la solución mediante microservicios independientes comunicados a través de un API Gateway.
- Documentar APIs, exponer health checks y entregar pruebas unitarias de reglas de negocio.

3. Modalidad de la Prueba

- Trabajo en equipo de 2 personas, preferiblemente un perfil Senior Developer y un perfil Analyst / Developer.
- Tiempo de duración: 6 horas.
- La solución debe enfocarse en arquitectura, reglas de negocio, integración externa, consistencia de datos y buenas prácticas.
- No se requiere integración con exchanges para compra o venta de criptomonedas.
- No se requiere autenticación compleja, KYC, billeteras digitales ni manejo de llaves privadas.

4. Alcance Funcional

4.1 Consulta de Mercado de Criptomonedas

El módulo de datos de mercado debe permitir consultar información pública de criptomonedas desde una API externa. Este módulo debe implementarse como un microservicio independiente responsable de consumir, normalizar y cachear información de mercado.

El microservicio de datos de mercado debe incluir como mínimo:

- Cliente HTTP para consumir API externa.
- Normalización de la respuesta externa a DTOs internos.
- Uso de Memory Cache nativo de .NET para consultas frecuentes.
- Manejo de errores cuando la API externa no responda.
- Endpoint de health check que valide disponibilidad de la API externa.
- Documentación Swagger/OpenAPI.

La consulta de mercado debe mostrar como mínimo:

Campo	Descripción
Nombre	Nombre de la criptomoneda.
Símbolo	Símbolo de mercado, por ejemplo BTC.
Precio actual USD	Último precio obtenido desde la API externa.
Variación 24h	Porcentaje de variación en las últimas 24 horas.
Volumen 24h	Volumen reportado por la API externa.
Market Cap	Capitalización de mercado.
Última actualización	Fecha y hora de la última consulta válida.

APIs externas sugeridas:

- CoinGecko REST API: endpoints como /coins/markets, /simple/price y /coins/{id}/market_chart.
- Binance Spot WebSocket Streams: alternativa para recibir datos públicos de mercado por stream.
- Se permite usar otra API pública siempre que el equipo documente endpoint, límites, manejo de errores y formato de respuesta.

4.2 Gestión de Criptomonedas Favoritas

La solución debe permitir que el usuario marque criptomonedas como favoritas para priorizar su visualización en el dashboard.

- Registrar una criptomoneda como favorita.
- Eliminar una criptomoneda de favoritos.
- Consultar lista de favoritas.
- Evitar duplicados para la misma criptomoneda.
- Persistir favoritas en SQL Server desde el microservicio correspondiente.

4.3 Gestión de Alertas de Precio

Registrar y editar una alerta indicando:

- Criptomoneda.
- Precio objetivo.
- Condición: mayor o igual que / menor o igual que.
- Moneda de referencia: USD.
- Observación.

Reglas de negocio:

- No se permiten precios objetivo menores o iguales a cero.
- No se puede crear una alerta para una criptomoneda inexistente o no soportada por la API externa.
- No se permite crear una alerta duplicada activa para la misma criptomoneda, condición y precio objetivo.

- No se puede editar una alerta en estado Cumplida o Anulada.
- No se puede anular una alerta en estado Cumplida.
- La anulación de alertas es lógica; se conserva el registro para trazabilidad.

4.4 Evaluación y Cumplimiento de Alertas

El sistema debe permitir evaluar alertas activas contra el precio actual de mercado. Esta evaluación puede ejecutarse de forma manual mediante endpoint o automáticamente mediante un proceso interno programado.

- Si la condición de la alerta se cumple, la alerta pasa a estado Cumplida.
- Debe registrarse un evento de cumplimiento con precio detectado, fecha, condición y origen de la evaluación.
- Una alerta solo puede cumplirse una vez.
- Si dos procesos intentan cumplir la misma alerta simultáneamente, solo uno debe completar la operación.
- La operación de cumplimiento debe ejecutarse dentro de una transacción usando Unit of Work.

4.5 Dashboard de Mercado y Alertas

Se requiere un dashboard que permita visualizar información de mercado y estado de alertas en tiempo real o tiempo cercano al real. La actualización debe ocurrir sin refresco manual de la página.

El dashboard debe mostrar:

- Top de criptomonedas monitoreadas.
- Precio actual.
- Variación 24h.
- Indicador visual de subida o bajada.
- Lista de criptomonedas favoritas.
- Alertas activas.
- Alertas cumplidas.
- Última hora de actualización.
- Estado de conexión con la API externa.

La actualización puede implementarse con SignalR, polling controlado cada 10 a 30 segundos o Server-Sent Events. El equipo debe justificar la opción elegida.

5. Aspectos Técnicos

5.1 Arquitectura de Microservicios (NO Monolito)

La solución debe estar estructurada como microservicios independientes. No se acepta arquitectura monolítica. Los servicios mínimos requeridos son:

Componente	Responsabilidad
MarketDataService	Consulta API externa, normaliza datos de mercado, aplica Memory Cache, expone endpoints de precios y salud de integración externa.
AlertasService	CRUD de alertas, reglas de negocio, Unit of Work, concurrencia, eventos de cumplimiento, acceso a SQL Server.
API Gateway	Punto único de entrada para el frontend. Debe implementarse con YARP u Ocelot sobre .NET 9.
Frontend	ASP.NET MVC, Angular, JavaScript, AJAX o jQuery. Consume exclusivamente el API Gateway.

Cada microservicio debe cumplir con arquitectura Clean: Dominio, Aplicación, Infraestructura y Presentación/API. Cada microservicio debe desplegarse como un Container App independiente en Azure. El API Gateway actúa como el único punto de entrada externo.

```

/src
  /MarketDataService
    /Domain
    /Application
    /Infrastructure
    /Presentation
  /AlertasService
    /Domain
    /Application
    /Infrastructure
    /Presentation
  /ApiGateway
  
```

```

/Frontend
/tests
/AlertasService.Tests

```

5.2 Integración con APIs Externas

El microservicio MarketDataService debe consumir una API externa real o simulada de forma controlada. La integración debe manejar:

- Timeouts.
- Errores HTTP.
- Rate limits o respuestas no disponibles.
- Respuesta vacía o incompleta.
- Normalización de campos.
- Fallback a datos cacheados cuando sea técnicamente razonable.
- Logging de errores sin exponer detalles sensibles al frontend.

El equipo debe documentar qué API usó, endpoints consumidos, ejemplo de respuesta y estrategia de manejo de fallos.

5.3 Uso de Memory Cache nativo de .NET

Debe utilizarse Memory Cache nativo de .NET, por ejemplo IMemoryCache, sin necesidad de instalar servicios externos como Redis o Memcached.

- Cachear listado de criptomonedas monitoreadas.
- Cachear detalle de criptomoneda durante un tiempo definido, por ejemplo 30 a 60 segundos.
- Definir claves de caché claras, por ejemplo market:top:usd o market:coin:bitcoin:usd.
- Configurar expiración absoluta o deslizante según el caso.
- Invalidar o renovar caché cuando corresponda.
- No usar caché como fuente transaccional para el estado de alertas.

El objetivo del caché es reducir llamadas innecesarias a la API externa y mejorar tiempos de respuesta. La fuente de verdad para alertas, favoritas y eventos debe ser SQL Server.

5.4 Aplicación del patrón Unit of Work

El microservicio AlertasService debe aplicar el patrón Unit of Work para agrupar operaciones transaccionales. Este patrón debe utilizarse como mínimo en:

- Creación de una alerta.
- Edición de una alerta.
- Anulación lógica de una alerta.
- Evaluación y cumplimiento de alertas.
- Registro del evento de cumplimiento.

Ejemplo de operación transaccional esperada:

1. Iniciar Unit of Work / transacción.
2. Consultar alerta activa.
3. Consultar precio actual desde MarketDataService.
4. Validar si se cumple la condición.
5. Cambiar estado de alerta a Cumplida.
6. Registrar evento de cumplimiento.
7. Confirmar cambios.
8. Si algo falla, ejecutar rollback.

5.5 Control de Concurrencia

La solución debe manejar concurrencia adecuadamente. El caso mínimo requerido es evitar que una misma alerta sea marcada como Cumplida más de una vez cuando dos procesos intenten evaluarla simultáneamente.

Escenario obligatorio:

```

Alerta: BTC >= 100000 USD
Estado inicial: Activa
Proceso A detecta precio BTC = 101000 USD
Proceso B detecta precio BTC = 101000 USD
Resultado esperado: solo un proceso marca la alerta como Cumplida y solo se registra un evento de cumplimiento.

```

Se permite resolverlo mediante una de las siguientes estrategias:

- Optimistic locking usando RowVersion en SQL Server.
- Transacción con bloqueo sobre la alerta, por ejemplo UPDLOCK.
- Índice único o restricción que impida más de un evento de cumplimiento por alerta.
- Combinación de RowVersion + transacción + validación de estado.

Ante conflicto de concurrencia, la API debe responder con 409 Conflict o error de negocio equivalente.

5.6 Health Check de Base de Datos y API Externa

Cada microservicio debe exponer un endpoint de health check. El health check debe validar al menos:

- Estado general del microservicio.
- Conexión a SQL Server cuando aplique.
- Disponibilidad de la API externa desde MarketDataService.
- Estado del caché, si se desea reportar.

```
GET /health -> { "status": "Healthy", "database": "Connected", "externalApi": "Available" }
```

5.7 APIs REST

Todos los endpoints deben estar documentados con Swagger/OpenAPI/Scalar y ser accesibles a través del API Gateway.

Método	Ruta Gateway	Descripción
GET	/api/market/coins	Consultar listado de criptomonedas monitoreadas.
GET	/api/market/coins/{id}	Consultar detalle de una criptomoneda.
GET	/api/market/coins/{id}/history	Consultar histórico básico de precio, opcional.
GET	/api/favorites	Consultar criptomonedas favoritas.
POST	/api/favorites	Registrar criptomoneda favorita.
DELETE	/api/favorites/{id}	Eliminar favorita.
GET	/api/alerts	Consultar alertas.
POST	/api/alerts	Crear alerta.
PUT	/api/alerts/{id}	Editar alerta.
DELETE	/api/alerts/{id}	Anular alerta lógicamente.
POST	/api/alerts/evaluate	Evaluar alertas activas.
GET	/api/health/market	Health check de MarketDataService.
GET	/api/health/alerts	Health check de AlertasService.

5.8 Unit Tests

El proyecto de AlertasService debe contar con un proyecto de unit tests independiente. Los tests deben cubrir reglas de negocio, validaciones, Unit of Work y concurrencia. Deben incluir 5 pruebas positivas y 5 pruebas negativas como mínimo.

Código	Caso de prueba
P1	Crear alerta válida para criptomoneda soportada.
P2	Editar alerta activa con nuevo precio objetivo válido.
P3	Anular alerta activa y conservar trazabilidad.
P4	Evaluar alerta y marcarla como Cumplida cuando se alcanza el precio objetivo.
P5	Consultar detalle de alerta con evento de cumplimiento registrado.
N1	Rechazar alerta con precio objetivo menor o igual a cero.
N2	Rechazar alerta para criptomoneda inexistente.
N3	Rechazar alerta duplicada activa.
N4	Rechazar edición de alerta Cumplida o Anulada.
N5	Simular evaluación concurrente y verificar que solo se genere un cumplimiento.

6. Entregables

- Diagrama de arquitectura: microservicios, API Gateway, SQL Server, frontend, API externa y flujo de datos.
- Diagrama paso a paso del proceso principal: consultar mercado, crear alerta y evaluar cumplimiento.
- Casos de uso del sistema.
- Prototipo de interfaz de usuario.
- Código fuente de todos los proyectos.

- Scripts de base de datos SQL Server.
- Implementación de Memory Cache nativo de .NET en MarketDataService.
- Implementación del patrón Unit of Work en AlertasService.
- Swagger/OpenAPI/Scalar funcionando.
- Health checks funcionando.
- Proyecto de pruebas unitarias.
- Evidencia del escenario de concurrencia.
- Solución publicada en Azure Container Apps, cuando aplique.
- README con instrucciones de ejecución local y despliegue.

7. Tabla de Evaluación

Tema	Pts.	Aspectos por calificar	Pts. Asig.
Análisis del Requerimiento	10	Presenta diagrama completo del problema: consulta de mercado, cache, creación de alerta, evaluación y cumplimiento.	5
		Genera los casos de uso del problema con flujos principales, alternos, excepciones y postcondiciones.	5
Planificación de la solución	10	Elaboración de prototipo y aplicación consistente en el producto finalizado.	10
Capa de presentación ASP.NET MVC, JavaScript, AJAX, jQuery o Angular	15	No utilización de otras tecnologías no permitidas.	5
		Cumple con todos los campos indicados en el requerimiento: mercado, favoritas, alertas y dashboard.	5
		Cumple con validaciones indicadas y actualización sin refresco manual.	5
Microservicios (Container Apps)	40	Diseño apropiado de arquitectura de microservicios. Cumple con arquitectura Clean: Aplicación, Dominio, Infraestructura y Presentación.	8
		Integración correcta con API externa de criptomonedas y manejo de errores.	6
		Uso correcto de Memory Cache nativo de .NET en endpoints de lectura.	5
		Aplicación correcta del patrón Unit of Work en operaciones transaccionales.	6
		Exposición correcta de APIs REST mediante API Gateway.	7
		Acceso a datos SQL Server y uso de LINQ / Dapper / Entity Framework.	8
Buenas prácticas en desarrollo de Software	10	Realización de unit tests, incluyendo casos positivos, negativos y concurrencia.	6
		Código limpio, documentado, manejo de errores, logs y respuestas HTTP adecuadas.	4
Eficiencia y Gestión del Tiempo	15	Entrega menor a 4 horas.	15
		Entrega entre 4 y 6 horas.	10
		Entrega mayor a 6 horas.	5
TOTAL	100		100

8. Reglas para considerar

- La prueba debe ser desarrollada exclusivamente por el equipo asignado, sin recibir apoyo externo no autorizado.
- No se permite convertir el ejercicio en una plataforma real de trading, compra, venta o custodia de criptomonedas.
- El frontend debe consumir exclusivamente el API Gateway.
- Los microservicios no deben exponerse directamente al exterior.
- El API Gateway no debe contener reglas de negocio.
- La validación crítica de reglas de negocio debe realizarse en backend.
- El caché no debe reemplazar la persistencia de alertas ni eventos.
- La solución debe manejar errores de la API externa sin detener completamente la aplicación.
- Se debe mantener trazabilidad de alertas anuladas y cumplidas.

9. Anexo A - Casos de uso mínimos esperados

CU-01 Consultar mercado de criptomonedas

Actor principal	Objetivo	Precondición	Postcondición
Usuario administrador	Visualizar información actualizada de mercado.	La API externa está configurada.	El usuario visualiza datos normalizados y cacheados.

Flujo principal:

1. El usuario ingresa al dashboard.
2. El frontend solicita listado de criptomonedas al API Gateway.
3. El API Gateway enruta a MarketDataService.
4. MarketDataService verifica si existen datos en Memory Cache.
5. Si existe caché válido, responde desde caché.
6. Si no existe caché válido, consulta la API externa, normaliza la respuesta y almacena en caché.
7. El dashboard muestra precio, variación, volumen y última actualización.

Excepciones / flujos alternos:

- API externa no disponible.
- Respuesta incompleta.
- Timeout.
- Caché expirado sin respuesta externa.

CU-02 Crear alerta de precio

Actor principal	Objetivo	Precondición	Postcondición
Usuario administrador	Registrar una alerta sobre una criptomoneda.	La criptomoneda existe en la API externa.	La alerta queda registrada en estado Activa.

Flujo principal:

1. El usuario selecciona una criptomoneda.
2. El sistema muestra el precio actual.
3. El usuario ingresa condición, precio objetivo y observación.
4. El sistema valida precio mayor a cero.
5. El sistema valida que no exista alerta activa duplicada.
6. AlertasService registra la alerta usando Unit of Work.
7. El sistema muestra la alerta en el dashboard.

Excepciones / flujos alternos:

- Precio inválido.
- Criptomoneda inexistente.
- Alerta duplicada.
- Error de base de datos.

CU-03 Editar alerta de precio

Actor principal	Objetivo	Precondición	Postcondición
Usuario administrador	Modificar condición, precio objetivo u observación.	La alerta existe y está Activa.	La alerta queda actualizada.

Flujo principal:

1. El usuario selecciona una alerta activa.
2. El sistema muestra sus datos actuales.
3. El usuario modifica precio, condición u observación.
4. El sistema valida las reglas de negocio.
5. AlertasService actualiza la alerta usando Unit of Work.
6. El dashboard refleja el cambio.

Excepciones / flujos alternos:

- Alerta Cumplida.
- Alerta Anulada.
- Precio menor o igual a cero.
- Alerta duplicada resultante.

CU-04 Anular alerta

Actor principal	Objetivo	Precondición	Postcondición
Usuario administrador	Anular lógicamente una alerta activa.	La alerta existe y está Activa.	La alerta queda en estado Anulada.

Flujo principal:

1. El usuario selecciona una alerta.
2. El sistema solicita confirmación.
3. El usuario confirma.
4. AlertasService valida que la alerta esté Activa.
5. AlertasService cambia estado a Anulada usando Unit of Work.
6. El dashboard actualiza la lista de alertas.

Excepciones / flujos alternos:

- Alerta inexistente.
- Alerta Cumplida.
- Alerta ya Anulada.
- Error de concurrencia.

CU-05 Evaluar alertas activas

Actor principal	Objetivo	Precondición	Postcondición
Sistema / Usuario administrador	Verificar si las alertas activas cumplen su condición.	Existen alertas activas.	Las alertas cumplidas quedan marcadas una sola vez y se registra evento.

Flujo principal:

1. El sistema inicia evaluación manual o programada.
2. AlertasService obtiene alertas activas.
3. Para cada alerta consulta precio actual por MarketDataService.
4. Valida si la condición se cumple.
5. Si se cumple, inicia Unit of Work.
6. Bloquea o valida RowVersion de la alerta.
7. Cambia estado a Cumplida y registra evento.
8. Confirma transacción.
9. Notifica visualmente al dashboard.

Excepciones / flujos alternos:

- API externa no responde.
- Precio no disponible.
- Conflicto de concurrencia.
- Alerta ya fue procesada por otro proceso.

10. Anexo B - Modelo de datos sugerido

Tabla	Campos principales
MonedasFavoritas	Id, CoinId, Symbol, Name, IsActive, FechaCreacion

Alertas	Id, CoinId, Symbol, TargetPrice, ConditionType, Currency, Status, Observation, CreatedAt, TriggeredAt, RowVersion
EventosAlerta	Id, AlertId, DetectedPrice, ConditionType, TriggeredAt, Source, Notes
MarketSnapshots	Id, CoinId, PriceUsd, Change24h, Volume24h, MarketCap, RetrievedAt. Opcional para trazabilidad.

Ejemplo SQL simplificado:

```
CREATE TABLE Alertas (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    CoinId NVARCHAR(100) NOT NULL,
    Symbol NVARCHAR(20) NOT NULL,
    TargetPrice DECIMAL(18,8) NOT NULL,
    ConditionType NVARCHAR(20) NOT NULL,
    Currency NVARCHAR(10) NOT NULL DEFAULT 'USD',
    Status NVARCHAR(20) NOT NULL,
    Observation NVARCHAR(500) NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT GETDATE(),
    TriggeredAt DATETIME2 NULL,
    RowVersion ROWVERSION
);

CREATE TABLE EventosAlerta (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    AlertId INT NOT NULL,
    DetectedPrice DECIMAL(18,8) NOT NULL,
    ConditionType NVARCHAR(20) NOT NULL,
    TriggeredAt DATETIME2 NOT NULL DEFAULT GETDATE(),
    Source NVARCHAR(100) NOT NULL,
    CONSTRAINT FK_EventosAlerta_Alertas FOREIGN KEY (AlertId) REFERENCES Alertas(Id),
    CONSTRAINT UQ_EventosAlerta_AlertId UNIQUE (AlertId)
);
```

11. Anexo C - Estrategia recomendada de concurrencia

Se recomienda aplicar una combinación de validación de estado, transacción y restricción única para garantizar que una alerta se cumpla una sola vez.

1. Iniciar transacción desde Unit of Work.
2. Consultar la alerta con bloqueo o validar RowVersion.
3. Verificar que la alerta continúe en estado Activa.
4. Consultar o recibir el precio detectado.
5. Evaluar condición.
6. Actualizar estado a Cumplida.
7. Insertar evento de cumplimiento.
8. Confirmar transacción.
9. Si otro proceso ya cumplió la alerta, rechazar con 409 Conflict o ignorar de forma controlada.

Ejemplo conceptual SQL Server:

```
BEGIN TRANSACTION;

SELECT *
FROM Alertas WITH (UPDLOCK, ROWLOCK)
WHERE Id = @AlertId;

IF @Status <> 'Activa'
BEGIN
    ROLLBACK;
    THROW 50001, 'La alerta ya fue procesada o no está activa', 1;
END;

UPDATE Alertas
SET Status = 'Cumplida', TriggeredAt = GETDATE()
WHERE Id = @AlertId AND Status = 'Activa';

INSERT INTO EventosAlerta (AlertId, DetectedPrice, ConditionType, Source)
```

```
VALUES (@AlertId, @DetectedPrice, @ConditionType, @Source);
```

```
COMMIT;
```

12. Checklist de entrega

- Hay diagrama de arquitectura.
- Hay casos de uso completos.
- Hay prototipo de dashboard y gestión de alertas.
- Frontend consume únicamente API Gateway.
- MarketDataService consume API externa.
- MarketDataService usa IMemoryCache.
- AlertasService aplica Unit of Work.
- Existe control de concurrencia para alertas cumplidas.
- Existe SQL Server y scripts de base de datos.
- Existen endpoints REST documentados.
- Existen health checks.
- Existen unit tests positivos y negativos.
- Existe prueba de concurrencia.
- La solución maneja errores de API externa.
- El README explica ejecución local, configuración de API externa y despliegue.